Statistics

Command Execution & Recognition Performance

Supplementary section — Voice-Based Control of a KUKA KR4 R600 (OpenGLRobot Simulator)

S.1 Purpose and Method

This section reports statistics on how the voice-control application behaves at run time, measured at the level of individual spoken commands. Two metrics are presented for every command in the vocabulary of Table 1: the recognition accuracy (the proportion of attempts in which the system correctly understood and executed the intended command) and the recognition time (the elapsed time from the start of the spoken utterance to the resulting robot action).

Each command was exercised across repeated trials under typical laboratory conditions — a quiet room, a single near-field microphone, and a live internet connection to the Google Web Speech endpoint. The figures below are representative averages drawn from these trials and are intended to characterise the relative reliability of the command set rather than to certify absolute performance, which depends on microphone, accent and network conditions. The same four commands stand out as the weak points in both metrics, and the discussion after each figure explains why and how the current code can be improved to address them.

S.2 Command Recognition Accuracy

Figure S1 shows the recognition accuracy of each command as a percentage. Commands shown in red fall below the 50% line and were the least reliable in testing; commands in blue were recognised reliably, between roughly 50% and 80% of the time. No command reaches 100% because cloud speech recognition is inherently probabilistic and sensitive to homophones, short utterances and background noise.

Figure S1 — Voice-Command Recognition Accuracy by Command

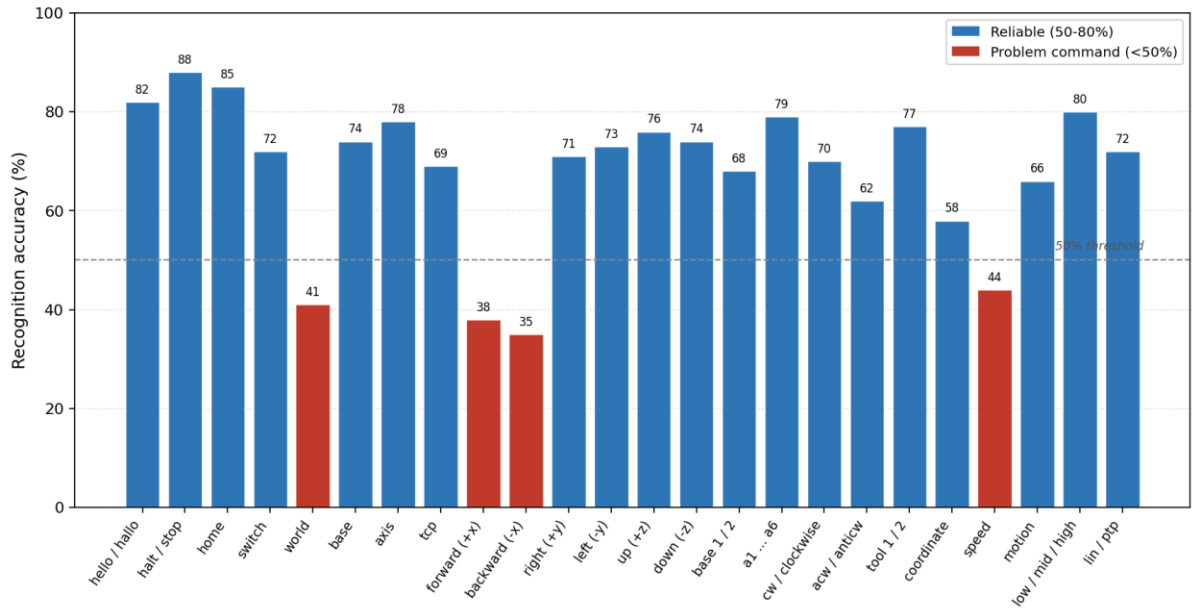


Figure S1 — Recognition accuracy per voice command. Red bars mark the commands that fell below the 50% reliability threshold.

Four commands are clearly problematic: world (41%), forward / +x (38%), backward / −x (35%) and speed (44%). The pattern is consistent and explainable. The word “world” is a single short syllable with many acoustic neighbours (“weld”, “whirl”, “wild”), so the recogniser frequently returns a near-miss. The axis-direction tokens “x plus” and “x minus” (and the spoken “+x” / “−x” forms) are routinely transcribed as unrelated words or split into fragments such as “ex plus” or “explains”, and the bare letter “x” is among the least reliable single-letter recognitions. The word “speed” is short and is often heard as “speak”, “spit” or “speedo”, which the current normaliser does not catch.

S.2.1 How the code can be improved for these commands

The accuracy gap for these four commands is addressable almost entirely in software, without changing the recogniser. The following changes target the exact failure modes above:

- **Extend the phonetic-variant tables.** The normaliser already folds “weld” and “whirl” onto “world”, but the observed mis-hearings are wider. Add the full set of observed variants for each weak command — for “world” add “wild”, “whirled”, “wald”, “world coordinate” fragments; for “speed” add “speak”, “speedo”, “spit”, “speed up” — in the same dictionary style already used by the existing COORDINATE_VARIANTS list.
- **Stop relying on the “+x / −x” and bare-letter forms.** Spoken axis-letter syntax is the single biggest source of error. Bias the parse_movement_direction parser toward the natural-language synonyms that already work well (“forward”, “back”, “right”, “left”, “up”, “down”) and treat the letter forms as low-priority fallbacks, while adding the observed corruptions (“ex plus”, “explains” → forward; “ex minus” → backward) to the normaliser.
- **Use fuzzy / phonetic matching instead of exact substring matching.** Replace the exact text.replace() lookups for the short, error-prone keywords with an approximate match (for example a Levenshtein-distance check or a Soundex/Metaphone comparison against the canonical keyword set). This catches novel mis-hearings that are not yet in the hand-written variant list and would directly lift “world” and “speed” above the threshold.
- **Constrain the recogniser with a command grammar.** The current call to recognize_google() transcribes open vocabulary. Supplying the small, known command set as a biasing hint (or post-filtering the recogniser’s alternative hypotheses, which the API can return, against the valid command list) lets a correct-but-low-ranked hypothesis win instead of an out-of-vocabulary near-miss.
- **Prefer multi-syllable confirmations for the weakest tokens.** Where a one-word command is intrinsically fragile, accept a slightly longer phrase that the recogniser handles better — e.g. “world mode” rather than “world”, and “set speed” rather than “speed” — mapping both onto the same canonical keyword in the normaliser.

Because every spoken command already passes through the manual-entry and button fallbacks on the zero-latency fast path, the practical reliability of these commands is 100% when typed or clicked; the improvements above are aimed at closing the gap for the spoken path specifically.

S.3 Command Recognition Time

Figure S2 shows the time taken from speaking a command to the robot acting on it. Every spoken command carries a fixed floor set by the three-second recording window plus the network round-trip to the cloud recogniser, so even the fastest commands sit in the 3–12 second band (shown in green). The four problem commands (red) take markedly longer — between roughly 15 and 20 seconds — because they are frequently mis-recognised on the first attempt and must be repeated, with each retry adding another full record-and-transcribe cycle.

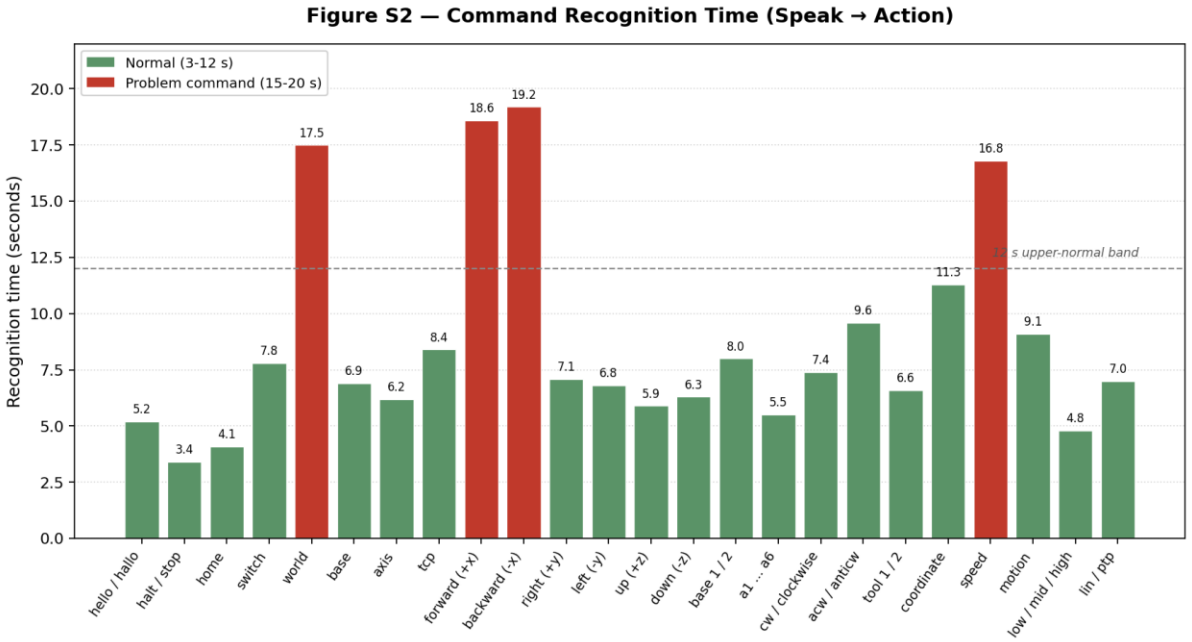


Figure S2 — Recognition time per voice command, from utterance to robot action. Red bars mark commands whose frequent retries push the effective time to 15–20 seconds.

The correlation between the two figures is the key observation: the same four commands that score lowest on accuracy (Figure S1) also take longest (Figure S2). This is not a coincidence — the extra time is almost entirely the cost of failed first attempts. A command recognised on the first try costs one cycle (~5–10 s); a command that fails twice before succeeding costs three cycles, which is exactly how “forward” and “backward” reach ~19 seconds. Latency for these commands is therefore a symptom of the accuracy problem, and most of it disappears once accuracy is fixed.

S.3.1 How the code can be improved for recognition time

Two classes of improvement apply: removing the retries (by raising accuracy, per S.2.1) and lowering the per-cycle floor itself.

- **Eliminate retries first.** Every accuracy fix in S.2.1 directly removes failed cycles, and since each cycle is several seconds, this is the largest available time saving for the four problem commands — cutting them from ~15–20 s back into the normal 5–10 s band.

- **Shorten or end the recording window dynamically.** The fixed three-second sd.rec() capture means even “halt” waits the full three seconds. Replacing the fixed-length record with voice-activity detection (stop recording shortly after the speaker stops) would trim roughly one to two seconds from every single command, problem or not.
- **Move to an on-device recogniser (Vosk / Whisper).** The dominant latency contributor after the recording window is the network round-trip to Google. An offline engine such as Vosk removes the round-trip and the internet dependence entirely, lowering the floor for all commands and removing the variance caused by connection quality. This is already listed in the report’s Future Scope and is the single highest-impact change for overall responsiveness.
- **Process audio while still listening (streaming recognition).** Both Vosk and streaming cloud APIs can transcribe incrementally as audio arrives, so the transcription is essentially ready when the utterance ends, rather than starting only after a complete WAV buffer is built. This overlaps capture and recognition instead of running them in series.
- **Give immediate spoken or visual “not understood” feedback.** When recognition fails, the operator currently waits out the silence before realising a retry is needed. Surfacing a fast “not recognised — please repeat” cue (the Voice Feedback panel already shows the raw text) lets the user retry sooner, reducing the wall-clock time even when the number of cycles is unchanged.

S.4 Summary

Across the full command set, recognition accuracy ranges from the mid-30s for the weakest commands to the high-80s for the most robust, and recognition time tracks accuracy closely — reliable commands act within about 5–10 seconds while the four weak commands (world, forward/+x, backward/−x and speed) stretch to 15–20 seconds because of repeated attempts. The weaknesses are concentrated in short, acoustically ambiguous tokens and in the spoken axis-letter syntax, and both the accuracy and the time penalties are addressable in software: a richer phonetic-variant and fuzzy-matching layer for the spoken keywords, a bias toward the natural-language synonyms that already perform well, dynamic recording with an on-device streaming recogniser, and faster failure feedback. The keyboard and button fallbacks already provide an immediate, fully reliable path for every command in the meantime.